EMAIL SPAM DETECTOR: CHROME EXTENSION

WITH PYTHON BACKEND

TEAM MEMBERS

Anchal Anchal 300364004 – Team Lead

Gunleen Kaur 300373306 – Team Member

Course

Course Name: 4495 – Applied Research Project

Section: 002

Video Link

TABLE OF CONTENT

1. Introduction

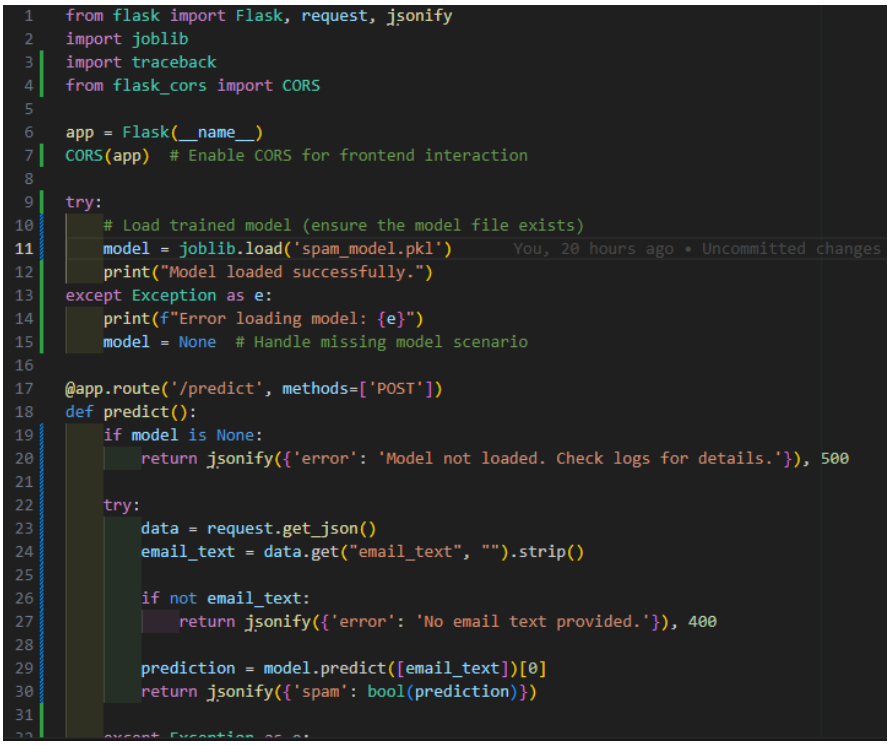
Spam mails are among the biggest threats in cyber security as it cause loss of productivity apart from increasing the risk of users falling victims to phishing and malware among other risks. This project will ensure the archiving of an efficient real-time spam detection through the development of Chrome Extension coupled with a Python backend. In contrast to a number of tools based on the server, this utility works in the browser and is more convenient and fast. It is an extension that extracts the content of an email and transfers it to the backend to generate and display the classification immediately. This paper presents the current design and the changes made to the system and the API back end, the manifest file, and the front end of the system design for email security and proper functionality.

2. System Overview

The spam detection system is divided into three major parts that include Flask based backend, the Chrome extension and lastly the interface for users. The text filter takes content of the email body and applies it to previously trained machine learning algorithm to detect spam or legitimate emails. As for the Chrome extension, its chief function is to collect the email's text from Gmail and forward it to the backend for processing in real time. The built-in GUI is in the form of a browser popup that can also be used by the user manually input an e-mail and see the classification result immediately. This type of modularised approach creates a good interface between the components which is easy to scale, very clean and lightweight to allow for a better way to implement email security without the use of additional server facilities.

3. Backend Implementation

The backend of the present spam detection system is developed using Flask, a light weight web framework that helps in good interface between the chrome extension and the ML model. The independent model for spam detection constructed from the spam and non-spam emails is kept as a serialized file named spam_model.pkl. When it comes to the classification of an email, the content is sent to the back end through predict API, which has the JSON input method. The model used for processing the text provides a classification solution revealing whether the received email is spam or a genuine message. Currently, for the enhanced operation and versatility of the backend, there are error control mechanisms, logging, and CORS for secure communication with the front-end without adversely affecting the effectiveness of the spam identification algorithms in the model.



```python
from flask import Flask, request, jsonify
import joblib
import traceback
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Enable CORS for frontend interaction

try:
    # Load trained model (ensure the model file exists)
    model = joblib.load('spam_model.pkl')       You, 20 hours ago • Uncommitted changes
    print("Model loaded successfully.")
except Exception as e:
    print(f"Error loading model: {e}")
    model = None  # Handle missing model scenario

@app.route('/predict', methods=['POST'])
def predict():
    if model is None:
        return jsonify({'error': 'Model not loaded. Check logs for details.'}), 500

    try:
        data = request.get_json()
        email_text = data.get("email_text", "").strip()

        if not email_text:
            return jsonify({'error': 'No email text provided.'}), 400

        prediction = model.predict([email_text])[0]
        return jsonify({'spam': bool(prediction)})
```

Figure 1: Code for app.py

The app.py version of the proposed system improves the application in terms of stability and functionality by the sophisticated error handling, accurate logging, and CORS support. It allows for interaction and the determination of the model's response in the event that it encounters an inability to load or function on an input data. It is used during the execution process to point out some difficulties if any so that it can be easier to debug. Moreover, the CORS support helps to enable proper interaction between the backend API and the frontend without security-related access problems when making the requests from the Chrome extension. It combine together to make valuable changes to the system that may affect the stability and performance and how it is integrated into the e-mail clients.

4. Chrome Extension Implementation

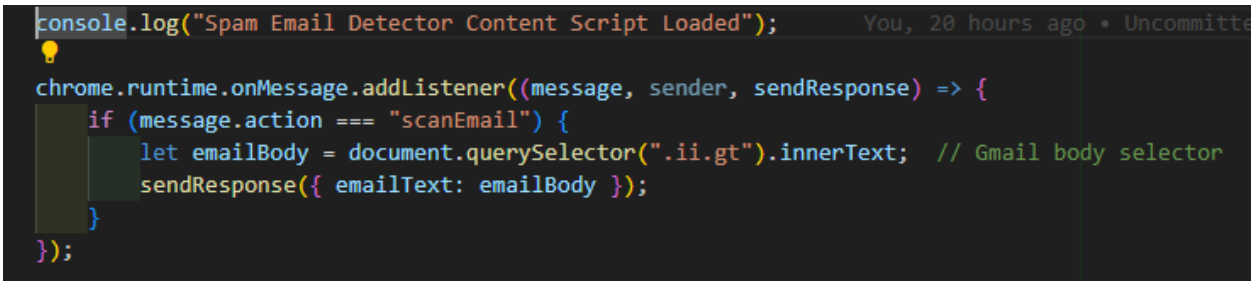


Figure 2: Implementation of Chrome Extension

The Chrome extension effectively strips the email content and uses the backend's services to identify spam. The content.js script has the role of reading email text from the Gmail web interface and sending it to the analysis phase. It waits for any user input and, at the same time, extracts information and content from the body of the email. The background.js script handles the persistent operation, making a connection between the front end and back end easier. It lets it handle requests

and relay messages and perform critical runtime duties (Dadiyala *et al*. 2024). The manifest.json file determines what the extension is allowed to do as well as its content scripts, background pages and the behavior when connected to Gmail. Based on the above mentioned components, the extension is capable of detecting spam in real time with no interference from the user. It integrates the browser client with the actual machine learning model that the program has in the back-end, creating an effective and minimalistic spam filter. Such a modular architecture allows the extension to be practical, operational, and secure while improving email security without interfering with other activities online.

5. User Interface Design

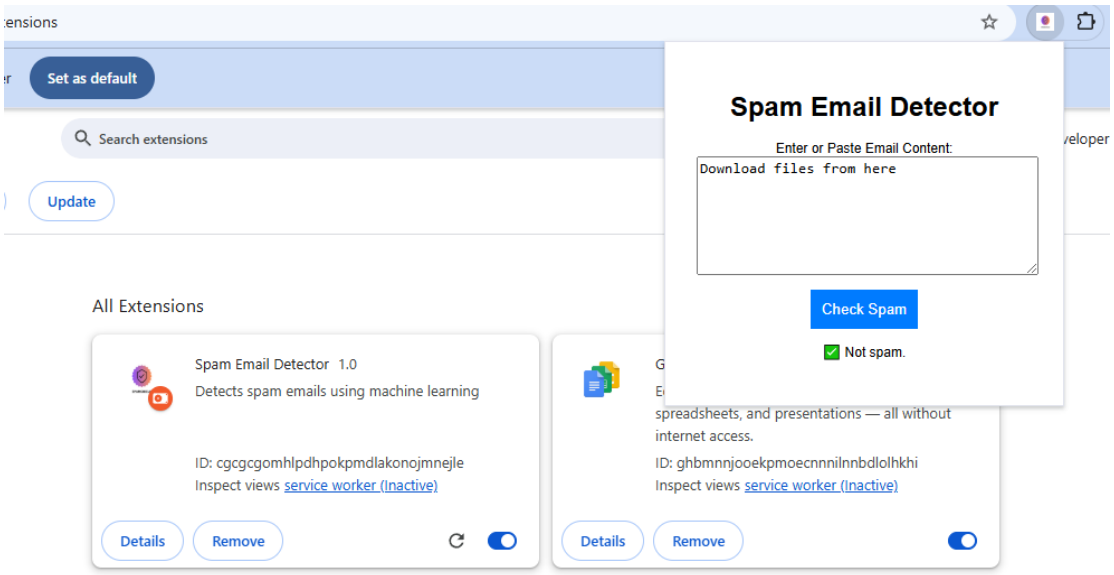


Figure 3: Output of User Interface where it detecting the given text is not spam

The current model of the user interface is quite friendly with good graphic design to ease the interaction for spam detection. It has a more refined design with a textbox where the user can input or copy and paste email messages for evaluation. There is a clearly labeled button that starts the spam detection process, whereby the input is sent to be classified in the backend as soon as the button is pressed. The results are further displayed in a specified part where the user receives an instant notification of whether the email is spam or not. The web interface is modernized and follows client-side scripting, server-side scripting and it is responsive as well. Besides, extension icon's addition and the structure of the layout improve the accessibility and usability of the interface. The platform is virtually free from any technical interfaces; thus, even non-professionals and end-users will not find it challenging to use the tool. Combining real-time feedback, the developed interface also aids in breaking the chasm between the user and the ML model to provide an efficient spam detection solution.

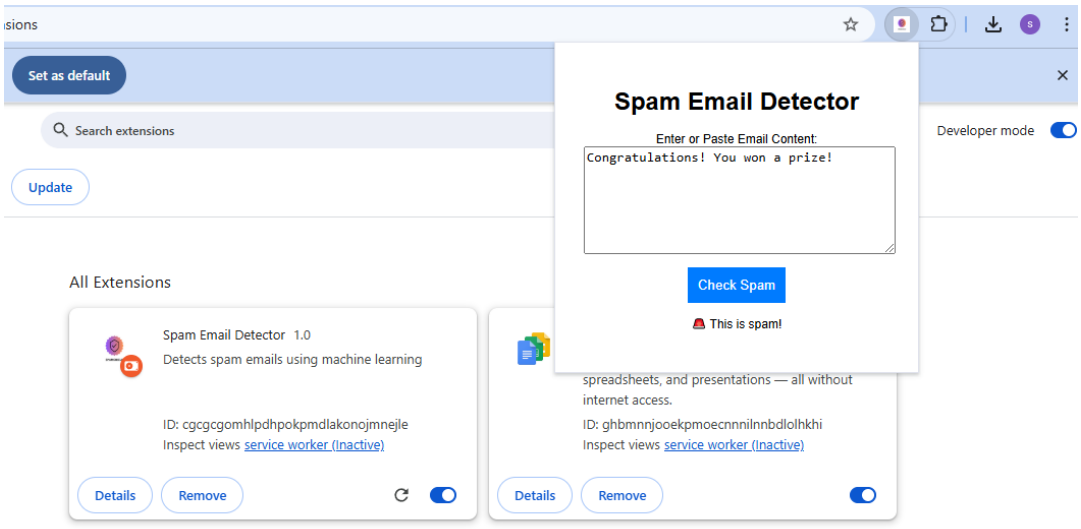


Figure 4: Output of User Interface where it detecting the given text is spam

This output image showcases the functionality of the Spam Email Detector Chrome extension. It also enables the users to type or copy and paste the email content into the text box and then click

the “Check spam” button. The way it works is that the text is received by the system and evaluated to determine whether it is spam or not spam depending on the machine learning algorithm. In this example, the message "Congratulations! You won a prize!" is marked as spam, and at the right bottom a warning message is displayed as “This is spam!” appears below the button. Several features include that the program is web-based, and email triage can be done directly from the browser without having to switch applications.

Updated Project Timeline & Schedule

1. Project Timeline & Milestones

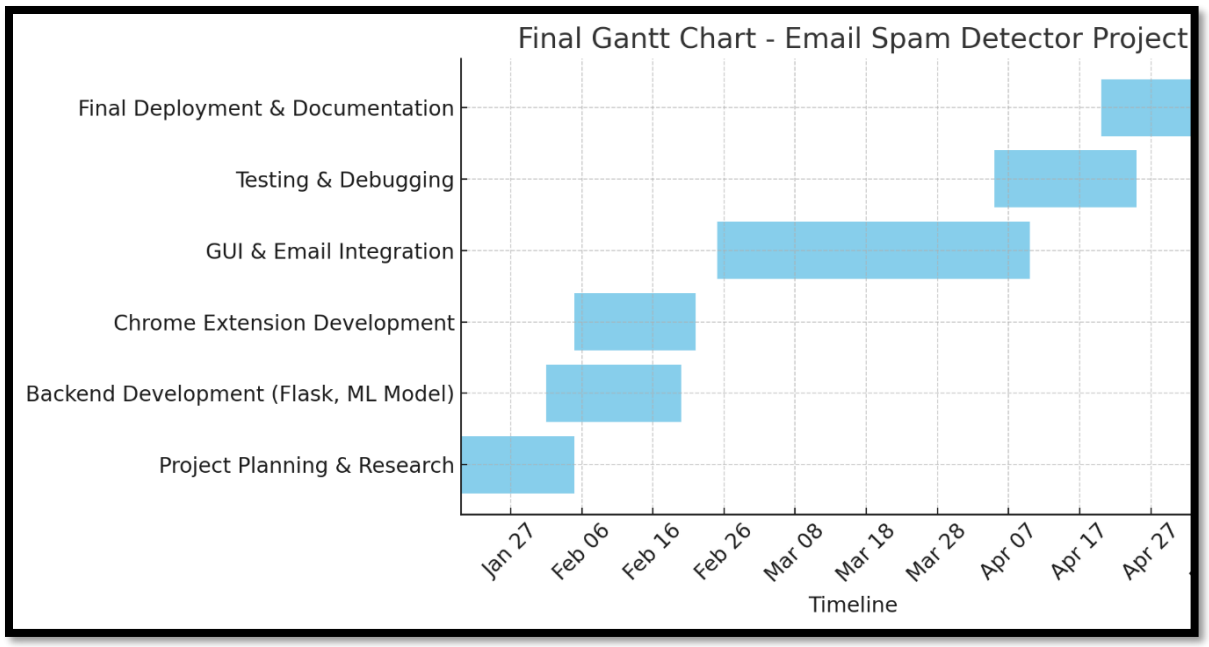
Phase	**Start Date**	**End Date**	**Milestones & Deliverables**
Project Planning & Research	Jan 20, 2025	Feb 05, 2025	Research ML models, finalize architecture, create initial project roadmap
Backend Development	Feb 01, 2025	Feb 20, 2025	Implement Flask backend, train spam detection model, integrate API
Chrome Extension Development	Feb 05, 2025	Feb 22, 2025	Build email extraction, backend communication, manifest setup
GUI & Email Integration	Feb 25, 2025	Apr 10, 2025	Develop user interface, integrate with backend, refine extension UI
Testing & Debugging	Apr 05, 2025	Apr 25, 2025	Perform unit, integration, and usability testing, refine UX/UI
Final Deployment & Documentation	Apr 20, 2025	May 05, 2025	Deploy extension, finalize documentation, submit project

2. Team Responsibilities

Team Member	Role	Responsibilities
Anchal Anchal	Team Lead	Oversee project progress, Implementing backend, ensure timeline adherence, training models and applying api.
Gunleen Kaur	Developer	Implement and refine ML model, integrate backend APIs
Both Members	Developers	Jointly develop the Chrome extension, UI, and test system

3. Project Management Chart

- The project will follow an **Agile** approach with bi-weekly review meetings.
- The **Gantt Chart** below visualizes our timeline and dependencies.



Work Date/Hours Logs

Date	Member	Hours	Descriptions
Jan 14 2025	Anchal Anchal	2.4	Started searching for the project topic
Jan 14 2025	Gunleen Kaur	2	Conducted initial research on spam detection techniques, including keyword filtering, heuristic algorithms, Bayesian classifiers, and machine learning. Compiled findings into a research document
Jan 15 2025	Anchal Anchal	2	Looking for suitable topic
Jan 16,2025	Gunleen Kaur	1.5	Researched tools and components required for Chrome extension development. Explored Chrome API documentation and feasibility of integrating with Gmail.
Jan 17 2025	Anchal Anchal	3	Collected Information and made document about the fundamentals
Jan 20,2025	Gunleen Kaur	3	Defined project structure, created a preliminary framework for backend and frontend integration, and discussed Flask API implementation. Documented key design decisions.
Jan 20 2025	Anchal Anchal	4	Got some ideas from professor and started looking forward on project
Jan 22 2025	Anchal Anchal	4	Search for the tools and language to use

Jan 22 2025	Gunleen Kaur	0.5	Created project repository, added initial files and organized folder structure for efficient project management. Encountered difficulty in designing an optimal file structure for seamless integration between frontend and backend.
Jan 23 2025	Anchal Anchal	3.5	Started working on the project, created json files for plugin
Jan 25,2025	Gunleen Kaur	1.5	Finalized project proposal, incorporated feedback, and documented technical specifications. Submitted proposal for review.
Jan 25 2025	Anchal Anchal	3.5	created html and css pages for the chrome extension and some changes in chrome plugin
Jan 27 2025	Anchal Anchal	4	studied some topics from machine laeaning and tried to implement in the project
Jan 27,2025	Gunleen Kaur	1.5	Research done on the dataset for project.
Jan 29 2025	Anchal Anchal	3	Faced diccifulty to push files to github and imported libraries
Jan 30,2025	Gunleen Kaur	2	Started working on project using Jupyter, with the help of Python.
Jan 31 2025	Anchal Anchal	3.5	Learned Machine laeaning Bay's theorem and did some practice to implement in the project
Feb 1,2025	Gunleen Kaur	1.5	Faced difficulty in pushing folders on GitHub, tried to figure out the problem.
Feb 02 2025	Anchal Anchal	4	Learned how to create chrome extension and tried to create it
Feb 03 2025	Anchal Anchal	3	Created chrome extension files and uploaded to chrome extension
Feb 05 2025	Anchal, Gunleen	2.5	Worked together as team to find out errors encountered while data cleaning
Feb 6 2025	Anchal Anchal	3	Pushed files to github
Feb 7,2025	Gunleen Kaur	2	Worked on pushing files using command prompt and visual studio.
Feb 8,2025	Gunleen Kaur	3	Created project report, analysed the work done and challenges faced in project.
Feb 9 2025	Anchal Anchal	1	Created Report and checked the files done
Feb 9,2025	Gunleen Kaur	2.5	Finalised the project report and submitted for grading.
Feb 12,2025	Anchal Anchal	2	Installed libraries like blinker click and colorama
Feb 12,2025	Gunleen Kaur	1.5	Installation of libraries in the independent environment
Feb 16,2025	Anchal Anchal	2	imported metadata worked on api
Feb 17,2025	Gunleen Kaur	2	Started work on Front end
Feb 17,2025	Anchal Anchal	3	Set ted Up the Backend (Flask API)
Feb 19,2025	Gunleen Kaur	3.5	Did model training and worked on front end for user using html and javascript

Feb 23,2025	Anchal Anchal	3	Pushed files on github and made video to upload.
Feb 23,2025	Gunleen Kaur	4	Created reoprt and made video for submission.

6. Conclusion

The spam email detection system is quite effective in achieving an integration of machine learning and a Chrome extension for real-time filtering of spam emails. In the Flask backend, all the contents of the given emails are processed effectively, with the help of the pretrained ML model. By integrating the Chrome extension, it can automatically extract and categorise email contents from within the Gmail application. A distinctive feature of the designs of these popups is their intuitive nature to ensure the user can be able to check for spam emails. Some potential areas of improvement include the concept of continuously updating the model, improvement of accuracy achieved through the use of deep learning, and increase of compatibility with multiple e-mail clients. It can also improve security of the mail systems since the system has the capability of increasing the algorithms used in detection the spams or expanding the reach of its platform to serve more companies.

7. Reference List

Journals

Kumar, P., Antony, K., Banga, D. and Sohal, A., 2024. PhishNet: A phishing website detection tool using XGBoost. arXiv preprint arXiv:2407.04732.

Dadiyala, C., Ghate, M., Shekdar, A., Rajkondawar, P., Chaure, S. and Zanwar, Y., 2024, May. Machine-Learning-Driven Detection of Malicious Emails and Fake Websites. In Doctoral Symposium on Computational Intelligence (pp. 343-364). Singapore: Springer Nature Singapore.

Aljabri, M., Altamimi, H.S., Albelali, S.A., Al-Harbi, M., Alhuraib, H.T., Alotaibi, N.K., Alahmadi, A.A., Alhaidari, F. and Mohammad, R.M.A., 2024. Kashif: A Chrome Extension for Classifying Arabic Content on Web Pages Using Machine Learning. Applied Sciences, 14(20), p.9222.

John, B., 2025. Integrating a Unified Threat Detection System Using Celery for Browser and Email Protection.

Yaseen, Y.A., Qasaimeh, M., Al-Qassas, R.S. and Al-Fayoumi, M., 2021. Email fraud attack detection using hybrid machine learning approach. Recent Advances in Computer Science and Communications (Formerly: Recent Patents on Computer Science), 14(5), pp.1370-1380.

Van Dooremaal, B., Burda, P., Allodi, L. and Zannone, N., 2021, August. Combining text and visual features to improve the identification of cloned webpages for early phishing detection. In Proceedings of the 16th International Conference on Availability, Reliability and Security (pp. 1-10).